

APUNTES

Víctor Manuel Peiró Martínez
FIIS 1ºA Arquitectura de Ordenadores

Contenido

Tema 1 – Introducción e Historia	3
1.1 – Conceptos Básicos	3
1.2 – Antecedentes Históricos	3
1.3 – Ordenadores Históricos	4
1.4 – Maquina de Turing	4
1.5 – Arquitectura Von Neumann	4
1.6 – Generaciones de Ordenadores	5
1.7 – Niveles de Abstracción	6
Tema 2 – Hardware.....	8
2.1 – Placa Base.....	8
2.2 – Memoria RAM	8
2.3 – Memoria ROM	9
2.4 – Almacenamiento	10
2.5 – Procesadores	12
Tema 3 – Principios del Algebra de Boole	13
3.1 – Algebra de Boole	13
3.2 – Puertas Lógicas.....	15
3.3 – Circuitos	17
3.3.1 – Circuitos Combinacionales.....	17
3.3.2 – Circuitos Secuenciales	20
Tema 4 – Aritmética	25
4.1 – Sistemas de Numeración	25
4.2 – Operaciones En Binario.....	27
4.3 – Conversión.....	31
4.3.1 – Binario y Decimal.....	31
4.3.2 – Octal y Decimal	32
4.3.3 – Hexadecimal y Decimal	32

Tema 5 – Funcionamiento de Computador.....	33
5.1 – Introducción.....	33
5.2 – Estructura General del Ordenador Programable.....	33
5.3 – Ejecución	34
5.4 – Ejemplo de Una Ejecución	35
5.5 – Diagrama de Estados de Un Ciclo	36
5.6 – Ciclo de Interrupción	36
6 – El Procesador.....	38
6.1 – Lenguaje Maquina	38
6.3.1 – Instrucciones Máquina.....	38
6.2 – Micro Z80.....	39
6.3.1 – Registros	39
6.3.2 – Pila	40
6.3 – Ensamblador Z80.....	41

Tema 1 – Introducción e Historia

1.1 – Conceptos Básicos

Ordenador: Dispositivo Programable, formada por circuitos electrónicos capaz de procesar información

- **Problemas:**
 - Variedad de Sistemas → Cada uno con sus características
 - Rápido Avance Tecnológico → Consistentes apariciones nuevas
- Surgen para automatizar cálculo y gestión de datos
- Inicialmente basados en tarjetas perforadas

Arquitectura de Ordenadores: El estudio del hardware de que equipos que conforman un sistema informático

- Integración de hardware y software
- Posibilidad de Escalado de los sistemas informáticos
- Abstracción para permitirlos al uso de usuarios no técnicos

1.2 – Antecedentes Históricos

Abaco: Permite realizar una serie de operaciones aritméticas

- Serie de cuentas colocadas en unos alambres que tienen un valor en función de su posición

La Pascalina: Serie de engranajes que permiten realizar una serie de operaciones aritméticas

- 8 dígitos en base decimal

Calculadora de Leibniz: Permite multiplicar y dividir simulando el desplazamiento que realizamos hacia la izquierda cuando multiplicamos a mano

Telar de Jacquard: Telar automático con tarjetas perforadas que indican el patrón de diseño a seguir en la confección

- Primera máquina programada

Máquina de Babbage: Máquina Diferencial capaz de construir tablas de logaritmos y trigonométricas

- Fallo por la tecnología para crear engranajes resistentes
- Concibe la idea de crear la maquina analítica
- **Ada Lovelace:** Introduce la algoritmia

Algoritmo: Al-Juarismi creador del concepto de los algoritmos y del álgebra.

Máquina Tabuladora: Creada para calcular el censo con tarjetas perforadas

- 300 tarjetas/min
- Al principio solo los gobiernos más ricos podían pagar la factura

1.3 – Ordenadores Históricos

No se puede atribuir a una persona la creación del ordenador.

Babbage: Padre del Ordenador

Torres Quevedo: Padre de la primera máquina de contar

Z3: Primera máquina programable y completamente autónoma

Mark I o Mark II: Primer ordenador electromecánico

- Basada en la máquina de Babbage
- Operaciones básicas y movimiento parabólico
- **Entrada de Datos:** Cintas Perforadas y Instrucciones con Tarjetas Perforadas
- **Bug:** Error de Programación

Colossus: Usada para leer comunicaciones cifradas

- Basada en los sistemas de Turing

ENIAC: Primera computadora de propósito general

- Electronic Numerical Integrator and Calculator
- Podía ser reprogramada

1.4 – Máquina de Turing

Turing: Desarrolla su máquina para descodificar el enigma (máquina alemana)

Contiene una cinta infinita, con un cabezal de lectura/escritura y un autómata móvil.

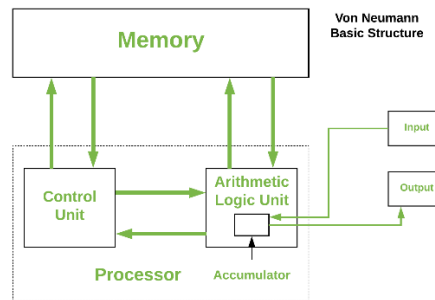
Puede resolver cualquier problema computable

Puede simular la lógica de cualquier algoritmo computable

1.5 – Arquitectura Von Neumann

Concepto del Programa Almacenado: Puede cambiar el cerebro a mi máquina

CPU: Conjunto entre ALU y el CU, conocido como el procesador



Componentes:

- **Unidad de Entrada y Salida**
- **Memoria:**
 - Compuesta por un conjunto de celdas de tamaño idéntico
 - Se almacenan datos e instrucciones
 - Se pueden acceder a estos datos mediante una dirección que las identifica
 - Las celdas se pueden leer y escribir
- **Unidad Aritmético Lógica (ALU):**
 - Utilizado para cálculos aritméticos y lógicos
 - Contiene los registros
- **Unidad de Control (CU):**
 - Encargada de leer instrucciones guardadas en la memoria y generar las señales de control necesarias para la misma
 - La instrucción actual está guardada en un puntero llamado el contador del programa (PC)

1.6 – Generaciones de Ordenadores

Primera Generación (1940-1952):

- Funcionan con válvulas de vacío
- Consume mucha energía.
- Tamaños considerables ya que se separan las válvulas para evitar el sobrecalentamiento
- Uso de tarjetas perforados y cintas magnéticas
- Solamente programables en lenguaje máquina

6.2 - Segunda Generación (1952 - 1964):

- Se sustituyen válvulas de vacío por transistores
- Hechos con silicio
- Más pequeño y barato
- Disipa mejor el calor

Tercera Generación (1964-1971)

- Aparición de la microelectrónica y circuitos integrados
- Permite sustituir a los circuitos fabricados con componentes discretos
- Aparecen puertas lógicas, celdas de memoria y circuitos de interconexión
- Almacenamiento a cabo de celdas de memoria
- Procesamiento de Datos: Realizado por puertas lógicas
- Transferencias de datos: La información
- Precio Invariable del chip

6.4 - Cuarta Generación (1971-1981)

- Se reduce tiempo de computación a 10 segundos
- Mas de 10000 puertas lógicas por chip
- Primer ordenador accesible para todo el mundo

6.5 - Quinta Generación (1981 – Actualidad)

- Procesamiento en paralelo y superconductores

1.7 – Niveles de Abstracción

Niveles:

- a. Elementos de entrada
- b. Elementos de salida
- c. Normas/ reglas para obtener los elementos de salida basándose en los elementos de entrada

Abstracción: Del nivel físico a las aplicaciones

Implementación: De aplicaciones al nivel físico

Nivel Físico:

- Material físico.
- Semiconductores como el silicio
- Permite controlar el flujo de los electrones por el
- Con él se construyen transistores, diodos, resistencias etc.

Nivel Electrónico

- Circuitos Electrónicos (transistores, diodos, resistencias etc.)
- Con ellos se crean puertas lógicas y biestables

Nivel Lógico

- Puertas Lógicas y Biestables conectados creando circuitos combinacionales y secuenciales
- Estos circuitos son multiplexores, contadores, sumadores, registros, ...

Nivel Transferencia de Registros

- Registros
- Circuitos Combinacionales y Secuenciales
- Elementos de interconexión que permite el intercambio de información entre los 2 primeros
- Salen conjuntos de transferencias elementales posibles en la ruta de datos construida por los 3 elementos anteriores

Lenguaje Máquina

- Frontera entre el hardware y el software
- Las transferencias se asocian con las instrucciones y el orden en las que se ejecutan
- Primer nivel en el que sus elementos básicos no son físicos
- Nivel queda definido como:
 - El repertorio de instrucciones
 - La memoria y el conjunto de registros

Software Básico (Sistema Operativo)

- Gestor de determinados recursos del nivel máquina que por frecuencia y complejidad de uso resulta más eficiente utilizarlos agrupados en una entidad que proporcione dichos servicios

Nivel Lenguajes de Alto Nivel

- Lenguajes de programación con una sintaxis parecida al lenguaje natural y alejado del lenguaje máquina
- Requiere traducción a nivel máquina por un compilador o intérprete

Algoritmos

- Resolución de un problema mediante un conjunto de reglas aplicadas de forma sistemática y ordenada
- Independientes de cualquier máquina o lenguaje de programación

Aplicaciones

- Nivel más alto
- Distintas actividades que se pueden resolver de forma automática

Tema 2 – Hardware

2.1 – Placa Base

BIOS: Permite arrancar de forma organizada en ordenador

- Basic Input Output System
- Etapa previa al SO
- Almacenado en el EEPROM y soportada por una pila CMOS
- Se utiliza para:
 - Configurar arranque
 - Configurar el reloj
 - Configurar los elementos de entrada/salida

Chipset: Conjunto de chips interconectados para el control de los componentes de la placa base

- Contiene un puente norte encargado de la memoria y un puente sur encargado de los dispositivos de entrada/salida
- Sistema causaba mucho cuello de botella
- **PCH:** Platform Controller Hub
 - Elimina el north y south bridge
 - Control de memoria y el PCI se asignan al CPU
 - PCH y CPU se unen mediante el DMI (Direct Media Interface)

Zócalos/Conectores: Permiten conexión de dispositivos de entrada/salida

- **Estándar actual:** SATA, aunque se sigue usando IDE
- Usados en tarjetas de sonido, video, red

PCI: Peripheral Component Interconnect

- Para conectar periféricos

2.2 – Memoria RAM

Random Access Memory

Almacena temporalmente instrucciones

Es memoria volátil (depende de una corriente eléctrica) de acceso aleatorio

Tipos de RAM: DDR5, DDR4, DDR3, ...

Conectadas al ordenador mediante zócalos específicos

Organización: Cuantos chips de capacidad conforman la memoria

Tiempo de Acceso: Para encontrar un dato, se selecciona el banco, luego la fila y finalmente la columna

Frecuencia: Frecuencia de acceso de la memoria

2.3 – Memoria ROM

Read Only Memory

No volátil (no depende de alimentación eléctrica)

No sirve como memoria principal ya que es muy cara, más lenta y tiene un límite máximo de escritura

Tipos:

- **ROM (Read Only Memory):** Solo Lectura, Contenido inalterable
- **PROM (Programmable Read Only Memory):** ROM que puede programarse una sola vez
- **EPROM (Erasable Programmable Read Only Memory):** ROM que se puede borrar por completo (mediante luz UV) después de ser reprogramada
- **EEPROM (Electrically Erasable Programmable Read Only Memory):** Puede ser borrada y modificada por software

2.4 – Almacenamiento

Todo equipo necesita almacenamiento no volátil para guardar programas

Primero fueron los discos magnéticos

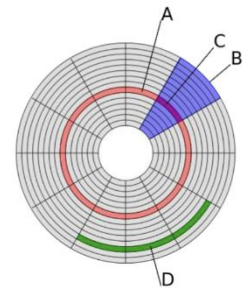
Disco Duro principal dispositivo de almacenamiento del PC

Pero se reemplazan por discos de estado sólido

Se siguen usando discos magnéticos en datacenters y como almacenamiento masivo y económico.

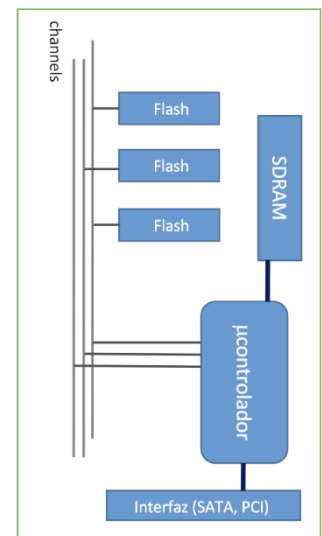
Disco Duro (HD):

- **Componentes:**
 - Disco Magnético
 - Cabezal de Lectura/Escritura
 - Actuador
- **Arquitectura:**
 - **A:** pistas
 - **B:** El disco se divide en sectores
 - **C:** La intersección entre el sector y el anillo se llama sector de anillo
 - **D:** Grupo consecutivo de sectores se llama clúster
- **Características:**
 - **Tiempo medio de acceso:** Tiempo que tarda en situarse la aguja en la pista y sector deseado
 - **Tiempo medio de búsqueda:** Tiempo que tarda la aguja en situarse en la pista deseada
 - **Tiempo de lectura/escritura:** Tiempo que tarda el disco en escribir/leer información
 - **Latencia Media:** Tiempo medio que tarda la aguja en situarse en el sector deseado
 - **Velocidad de Rotación:** Revoluciones por minuto de los platos
 - **Valores Típicos:** 5400, 7200 y 15000 rpm
 - **Tasa de Transferencia:** Velocidad a la que el ordenador puede transmitir información cuando la aguja está colocada



Unidad de Estado Solido (SSD):

- Memoria no volátil(flash) para almacenar datos
- Utilizan almacenamiento NAND flash
 - Mayor capacidad que las SDRAM, pero mayor tiempo de acceso
- Forma m.2: Permite velocidad de transferencia mayores
- **Ventajas:**
 - No tiene parte mecanicas
 - Mejor tiempo medio de fallos
 - No hay fragmentación
- **Desventajas:**
 - Menor vida útil
 - Se encuentran soluciones híbridas con SSDs para programas y HDs para datos
- **Arquitectura:**
 - **Microcontrolador:** Pequeña CPU que contiene la inteligencia de tu SSD
 - **SDRAM:** Memoria Auxiliar del microcontrolador
 - **Interfaz:** Comunicación con el driver
 - **Bancos de Memoria Flash:** Almacenan los datos
 - **Channels:** Buses para el intercambio de información entre el microcontrolador y los bancos de memoria
- SSD tiene su propio sistema de archivos FFS (Flash File System).
- **Wear Leveling:** Distribución de los ficheros en la memoria para no estresar unas pocas zonas



2.5 – Procesadores

Cerebro y corazón del PC

Su evolución se basa en la evolución de sus componentes

CPU (Central Processing Unit): Encargada de interpretar instrucciones del hardware

- **Características:**
 - **Frecuencia del Reloj:** Velocidad del procesador
 - **Consumo Energético**
 - **Numero de Núcleos:** Cuantos más núcleos, más tareas se pueden realizar simultáneamente
 - **Memoria Caché:** Memoria Local del Procesador
- **Límites de Las Prestaciones:**
 - **Potencia:** Los primeros micros no usaban disipadores. Ahora se necesitan disipadores, ventiladores y refrigeración líquida
 - **Frecuencia:** Frecuencia del reloj limitada a 4GHz, Limitada por el tiempo que se tarda en cargar y descargar los transistores
 - **Tensión de alimentación:** Se va disminuyendo poco a poco
 - **Potencia Estática:** A menor tensión aumenta la corriente de fugas de los transistores
 - **Número de Transistores:** Ver Ley de Moore
- **Ley de Moore:** El número de transistores se duplica cada 24 meses
 - La ley dejará de funcionar en 2025

GPU (Graphics Processing Unit): Da apoyo en la CPU en tareas sobre todo gráficas

Tema 3 – Principios del Algebra de Boole

3.1 – Algebra de Boole

Los ordenadores sólo entienden en binario

George Boole desarrolla una teoría matemática para manejar variable de sólo 2 valores

Definición: Para una variable x con $x = 0$ o $x = 1$

- **Existe la operación complemento:** $\bar{1} = 0$ y viceversa
- **Existe la operación producto lógico:**
 - $0 \cdot 0 = 0$
 - $0 \cdot 1 = 0$
 - $1 \cdot 0 = 0$
 - $1 \cdot 1 = 1$
- **Existe la operación suma lógica:**
 - $0 + 0 = 0$
 - $0 + 1 = 1$
 - $1 + 0 = 1$
 - $1 + 1 = 1$
- **El producto tiene preferencia sobre la suma**

Propiedades y Leyes de Morgan:

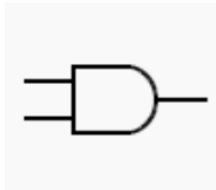
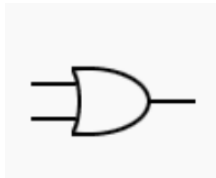
- **Identidad:**
 - $a + 1 = 1$
 - $a + 0 = a$
 - $a \cdot 1 = a$
 - $a \cdot 0 = 0$
- **Elemento Inverso:**
 - $\bar{\bar{0}} = 0$
 - $\bar{\bar{1}} = 1$
- **Idempotencia:**
 - $a + a = a$
 - $a \cdot a = a$
- **Involución:**
 - $\bar{\bar{a}} = a$
- **Asociativa:**
 - $a + (b + c) = (a + b) + c$
 - $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
- **Absorción:**
 - $a + (a \cdot b) = a$
 - $a \cdot (a + b) = a$

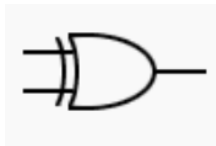
- **Leyes de Morgan:**
 - $\overline{a + b} = \bar{a} \cdot \bar{b}$
 - $\overline{a \cdot b} = \bar{a} + \bar{b}$
- **Propiedades Extra:**
 - $a + \bar{a} \cdot b = a + b$
 - $a \cdot (\bar{a} + b) = a \cdot b$

Representación de Sistemas Digitales:

- **Fórmula:** $x \cdot y$
- **Tablas de Verdad:** Tabla que muestra las salidas del sistema digital dependiendo de las entradas
- Circuitos lógicos dependientes de puertas lógicas

3.2 – Puertas Lógicas

Nombre	Símbolo	Función	Tabla De Verdad															
AND		$F = A \cdot B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F=A+B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = A$	<table><tr><th>A</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	

NAND		$F = A \cdot B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = A + B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR		$F = A \oplus B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Agrupación de Puertas Lógicas:

- Las puertas lógicas se pueden combinar para generar circuitos lógicos complejos
- Es posible generar puertas lógicas a partir de otras
- Algunas puertas, al agruparse componen un conjunto de puertas funcionalmente completo (mediante ellas puedo crear todas las funciones lógicas)
- Ejemplos:
 - AND, NOT, OR es el conjunto funcionalmente completo básico ya que contiene ya que tiene las tres funciones básicas
 - AND, NOT es el conjunto funcionalmente completo ya que podemos implementar OR con ellas
 - $A \text{ OR } B = \text{NOT}((\text{NOT } A) \text{ AND } (\text{NOT } B))$
 - NAND y NOR por sí solas son un conjunto funcionalmente completo ya que se puede implementar todas las puertas básicas

3.3 – Circuitos

Tipos:

- **Combinacionales:** Las salidas solo dependen del estado actual de sus entradas
- **Secuenciales:** Las salidas dependen del estado actual de sus entradas y del estado anterior

3.3.1 – Circuitos Combinacionales

Se componen de un conjunto de puertas interconectadas cuya salida depende del valor de las entradas en ese mismo instante

Retardo entre la aparición de salidas y entradas causado por el retardo de las puertas lógicas

Cada circuito combinacional de n entradas y m salidas se puede definir con:

- **Su tabla de verdad:** 2^n combinaciones de las n entradas con todos los valores de sus m salidas
- **Su símbolo:** El diagrama del circuito
- **Su función booleana:** Expresa los valores de la salida en función de sus entradas

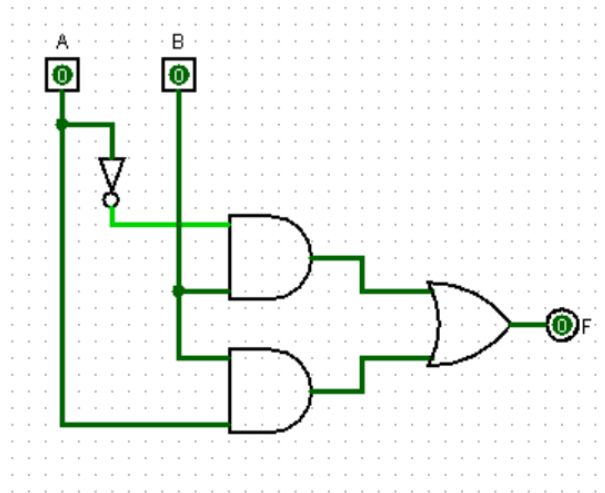
Implementación Según Tabla de Verdad:

1. Sacar función lógica. Si solo tenemos la tabla de verdad. Busco las combinaciones donde la salida sea 1.

A	B	F
0	0	0
0	1	1
1	0	0
1	1	1

$$F = \bar{A} \cdot B + A \cdot B$$

2. Creamos el circuito añadiendo al lado de una entrada su negada si son necesarios. Si las entradas se multiplican, puerta AND y si se están sumando, puerta OR.



Complejidad y Simplificación:

- La complejidad viene dada por la cantidad de puertas lógicas y conexiones que las componen.
- Se aconseja simplificar la función booleana y el circuito para obtener un circuito equivalente pero más sencillo.
- **2 formas de simplificación:**
 - **Simplificación Algebraica:** Utilizar las propiedades del Algebra de Boole para simplificar un circuito.
 - **Mapas de Karnaugh**

Mapas de Karnaugh:

- La simplificación se puede hacer de 2 maneras:
 - Suma de productos
 - Producto de Sumas
- **Suma de Productos:** Se dice que nuestra fórmula es una suma de productos (SOP) si está formada por productos relacionados por una suma.
 - Si tenemos la variable la codificamos como 1 sino como 0
 - **Pasos con Ejemplo:** $A + (B \cdot C)$

▪ Verificar si es SOP

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C$$

- #### ▪ Crear esqueleto:
- Para n entradas debe de haber 2^n celdas y entre celdas debe de haber solo un cambio en las variables

	00 $\bar{B}\bar{C}$	01 $\bar{B}C$	11 BC	10 $B\bar{C}$
0 $\bar{A}$				
1 A				

- Se coloca un 1 en cada celda donde la expresión aparezca en el SOP

	00 $\overline{B}\overline{C}$	01 $\overline{B}C$	11 BC	10 $B\overline{C}$
0 $\overline{A}$	0	0	1	1
1 A	1	0	0	1

- Se agrupan con valor 1 de tamaño sea 2^n (2, 4, 8, 16, ...) y se elimina la variable que no sea comun

	00 $\overline{B}\overline{C}$	01 $\overline{B}C$	11 BC	10 $B\overline{C}$
0 $\overline{A}$	0	0	1	1
1 A	1	0	0	1

- ABC y $ABC = AC$ porque B no es igual
- ABC y $ABC = AB$ porque C no es igual
- ABC y $ABC = BC$ porque A no es igual

$$F = A\overline{C} + \overline{A}B + B\overline{C}$$

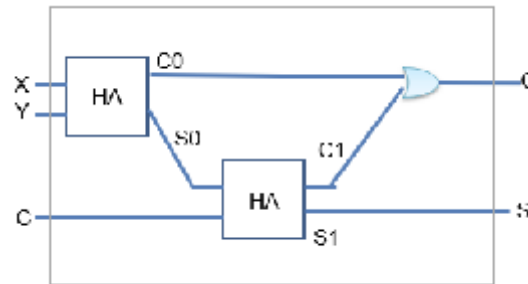
- Eliminar Conjuntos Solapados

$$F = A\overline{C} + \overline{A}B$$

- **Producto de Sumas:** Se dice que nuestra fórmula es un producto de sumas (POS) si está formada por sumas relacionadas por un producto.
 - Si tenemos la variable la codificamos como 0 sino como 1
 - Se aplica el mismo algoritmo, pero buscamos adyacentes con 0s
- **Circuitos Típicos:**
 - **Multiplexores:** Circuito con varias entradas y una sola salida.
 - Contiene un mecanismo de selección para ver qué entrada pasa a la salida
 - Las entradas de control(n) son potencias de 2. Entonces hay 2^n entradas de información
 - Para multiplexores de 8 o más salidas, se suelen utilizar multiplexores de 4 y 2 salidas
 - **Demultiplexores:** Misma función que los multiplexores, pero con salidas en vez de entradas. 1 entrada, n salidas
 - **Decodificadores:** Dispone de n entradas y 2^n salidas, dependiendo de la entrada se elige la salida

○ **Sumador:**

- **Semisumador:** Suma es una puerta XOR y acarreo una puerta AND
- Semisumador solo para 2 bits solo. Si queremos sumar más se emplea un sumador completo
- Para sumador completo, sumamos el resultado con el acarreo del anterior.
- El sumador completo se puede construir con 2 semisumadores



3.3.2 – Circuitos Secuenciales

Un circuito secuencial es aquel en el que sus salidas dependen del estado actual de sus entradas y de sus estados anteriores.

Se tiene en cuenta la evolución de señales temporales, es decir, que valores idénticos de entrada podrían tener salidas diferentes dependiendo del momento

La dependencia de las salidas es debida a la capacidad de “memorizar” información sobre los estados anteriores.

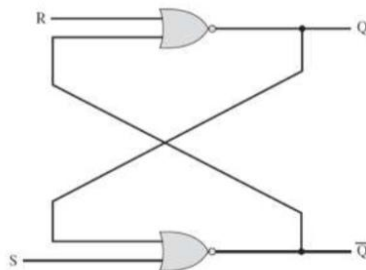
El Reloj: La señal reloj toma los valores 0 y 1 de forma continua y cíclica hasta que se detiene

- El ciclo de 0 y 1 tiene duración constante y determinada llamada el periodo. Se suele medir en nanosegundos
- La señal está formada por cristales de cuarzo ya que cuando reciben electricidad, vibran a grandes frecuencias.
- Cuando el reloj pasa de 0 a 1 se llama un flanco ascendente
- El intervalo entre un flanco y el siguiente es el ciclo del reloj.

Biestables: Circuitos capaces de “guardar” información

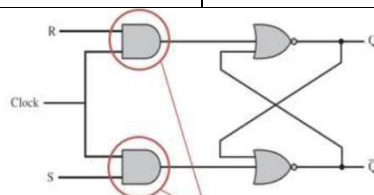
- Almacenan solo 1 bit
- Guardan su último estado hasta que cambian sus entradas
- **Características:**
 - Tiene 2 estados. Dependiendo del valor de sus entradas cambia de estado.
Si no hay cambio, permanece en el estado anterior
 - 2 salidas complementarias. Q y $\bar{Q}$
 - Dependen del tiempo
 - Estado Actual: Q_t
 - Estado Siguierte: Q_{t+1}
- **Biestable S-R:**
 - 2 entradas y 2 salidas
 - Asíncrono (no emplea reloj)
 - Sí, la entrada S es 1. Estado del circuito es 1
 - Si la entrada R es 1. Estado del circuito es 0
 - Si ambas son 1. No hay cambio ya que la entrada 1-1 no está definida

S	R	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	-



- **Biestable S-R Con Reloj**
 - Funciona igual que el S-R normal, pero si el reloj no está en 1, el circuito no está operativo

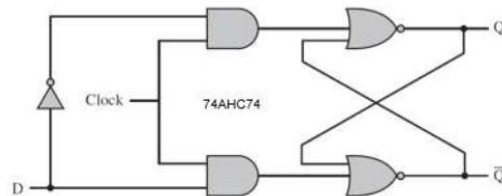
Clock	S	R	Q_{t+1}
1	0	0	Q_t
1	0	1	0
1	1	0	1
1	1	1	-
0	0	0	Q_t
...	...	...	...



- **Biastable D:**

- Síncrono (emplea Reloj)
- Una única entrada que elimina la entrada errónea 1-1
- La salida $t+1$ es igual a la entrada en t
- Cuando tenemos un biastable en un circuito, nos interesa que sea “insensible” a las variaciones de D a veces.
- Por eso se les añade una entrada de carga que si vale 0, el biastable no funciona. Si vale 1, funciona como debe.

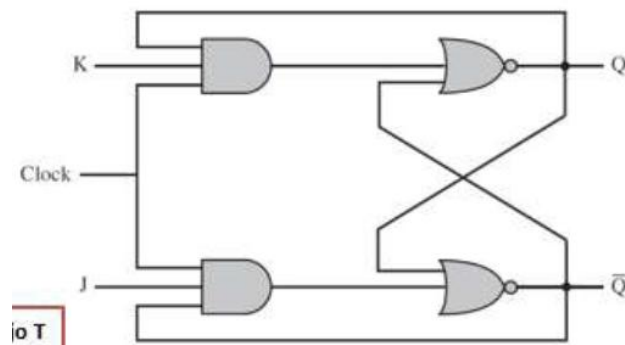
D	Clock	Q_{t+1}
0	0	Q_t
0	1	0
1	0	Q_t
1	1	1



- **Biastable J-K:**

- Síncrono (emplea reloj/los cambios se producen en los cambios activos del reloj)
- Similar a un S-R, pero admite todas las entradas
- La entrada 1 nos da la negación del estado anterior
- La entrada K puesta a 1 es un reset
- La entrada J puesta a 1 es un set

Clock	J	K	Q_{t+1}
1	0	0	Q_t
1	0	1	0
1	1	0	1
1	1	1	$\overline{Q_t}$



Registros:

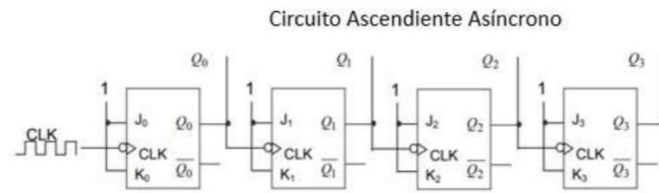
- Circuitos secuenciales capaces de almacenar varios bits
- Conjunto de biestables que funcionan en unísono compartiendo sus señales de información
 - La transferencia de bits de un registro puede ser:
 - En serie: Se transfieren 1 a 1
- En paralelo: Se transmiten todos simultáneamente utilizando tantas líneas como bits
- Según las conexiones entre los biestables, se obtiene un desplazamiento ó a la izquierda o a la derecha
- **Tipos:**

Entrada	Salida	Funcionamiento
Serie	Serie	La entrada y la salida se realiza bit a bit.
Serie	Paralelo	1 entrada y varias salidas. Se utiliza para conversión de serie a paralelo
Paralelo	Serie	1 salida y varias entradas. Se utiliza para la conversión de paralelo a serie
Paralelo	Paralelo	La entrada y salida se realizan por varios biestables

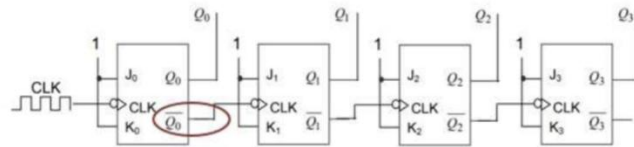
Contadores:

- Circuito secuencial que genera una secuencia ordenada de salidas que se repite con el tiempo
- La salida es el estado de sus biestables
- Utilizados para el control de fases de un proceso
- **Tipos:**
 - **Síncronos:** Todos los biestables comparten la misma señal de reloj
 - **Asíncronos:** No todos los biestables comparten la misma señal de reloj. Se aplica una señal de reloj a el primero y al resto se les aplica la salida del biestable anterior
 - **Ascendente:** Cuenta Creciente
 - **Descendente:** Cuenta decreciente
 - **Reversible:** Cuenta ascendiente o descendiente dependiente de una entrada de control

- **Contador Ascendente:**
 - Con biestables J-K
 - Señal de Q es la señal de reloj



- **Contador Descendente:**
 - Con biestables J-K
 - Señal de Q negada es la señal de reloj



Tema 4 – Aritmética

4.1 – Sistemas de Numeración

Sistema de Numeración: Conjunto de símbolos que se emplean para la representación de cantidades y las reglas que permiten trabajar con los mismos.

Sistema Posicional: Sistema de numeración en el cual la posición de los símbolos influye en su significado

Sistema Decimal:

- Sistema Posicional
- Base 10 - Símbolos 0-9
- Punto decimal establece que a su izquierda están los exponentes positivos y a la derecha están sus exponentes negativos
- **Representación:**
 - $1325 = 1 \cdot 10^3 + 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0$
 - $1,235 = 1 \cdot 10^0 + 3 \cdot 10^{-1} + 2 \cdot 10^{-2} + 5 \cdot 10^{-3}$

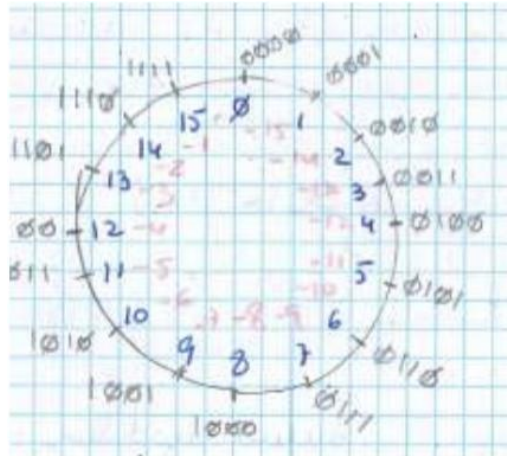
Sistema Binario:

- Sistema de numeración empleado por los ordenadores
- Cada dígito se denomina bit (binary digit)
- 8 bits forman un byte
- Base 2 - Símbolos 0 y 1
- **Unidades de la Información Digital:**

Para Almacenamiento (Bytes)			Para Transferencia(bit/s)	
Unidad	Símbolo	Valor en Base 2	Unidad	Símbolo
Byte:	B	2^0	Bit:	b
Kilobyte:	kB	2^{10}	Kilobit:	kb
Megabyte:	MB	2^{20}	Megabit:	Mb
Gigabyte:	GB	2^{30}	Gigabit:	Gb
Terabyte;	TB	2^{40}	Terabit:	Tb

- **Representación Modular:**

- Resta y Suma son la misma operación si la representación es adecuada
- Números equivalentes si: $A = B + KN$



Octal:

- Base 8
- Símbolos: 0-7

Hexadecimal:

- Base 16
- Símbolos: 0-9 y A-F
 - A=10, B=11, C=12, D=13, E=14, F=15

Teorema Fundamental de la Numeración: Relaciona un número en cualquier sistema posicional con su representación en decimal.

- $\sum X_i B^i$ con B = Base y X = Dígito
- **Ejemplo:** $543,3_6$ en base 10
 - $5 \cdot 6^2 + 4 \cdot 6^1 + 3 \cdot 6^0 + 3 \cdot 6^{-1} = 5 \cdot 36 + 4 \cdot 6 + 3 \cdot 1 + 3 \cdot 16 = 180 + 24 + 3 + 0.5 = 207.5$ en base 10

4.2 – Operaciones En Binario

Complemento a 2:

- Se cumple si $(k+p = 0) \bmod 2^n$
- Para convertir positivo a negativo, invierto bits y sumo 1
- Primer bit codifica en signo, pero la manera de codificar la magnitud varía así se asegura que el 0 solo tenga una representación.
- Rango asimétrico
- **Caja de Conversión a Decimal:**

-128	64	32	16	8	4	2	1

- Complemento a 1: Solo invierto bits

Cambio de Longitud:

- **Para S-M:** Añadir ceros entre el primer bit y el segundo hasta llegar a la longitud deseada
- **Para Ca2:**
 - Si negativo, rellenar con 1s al principio
 - Si positivo, rellenar con 0s

Representación de Decimales:

- **Representación en Punto Fijo:** Realizar potencias negativas y rellenar
 - **Limites:**
 - No se pueden representar números muy grandes
 - No se pueden representar fracciones muy pequeñas
- **Representación en Coma Flotante:**
 - Soluciona las limitaciones del punto fijo
 - Se separa en mantisa, base y exponentes
 - $M \cdot B^E$
 - **M:** Mantisa
 - **B:** Base
 - **E:** Exponente
 - Las mantisas han de estar normalizadas
 - El bit que acompaña a el signo es 1
 - Este bit es implícito y no se representa
 - **Ejemplo:** -0,28125 en binario y sesgo de 3 bits (3 bits para mantisa)
 - 1 001 0010
 - Signo: 1
 - $Exp = 1 - \text{sesgo} = 1 - 3 = -2$
 - Mantisa: 1001

- **IEEE 754:** Representación en coma flotante con características específicas
 - Precisión de 32 bits
 - 8 bits de Exponentes
 - 23 de Mantisa
 - 1 de signo
 - Exp: 0 y Mantisa: 0 $\rightarrow$ 0
 - Exp: 255 y Mantisa: 0 $\rightarrow$ +- infinito
 - Exp: 255 y Mantisa $\neq$ 0 $\rightarrow$ NaN

Negación: Obtener el negativo de un número. Depende de la representación

1. **En S-M:** Cambio el primer bit
2. **En Ca1:** Invierto bits
3. **En Ca2:** Hago Ca1 y sumo 1

Suma: Suma bit a bit

- **Reglas:**
 - $0+0 = 0$
 - $0+1, 1+0 = 1$
 - $1+1 = 0$ con acarreo 1
 - $1+1+1 = 1$ con acarreo 1
- **Desbordamiento:** Obtenemos un bit más al realizar la suma
- **Carry:** Hay desbordamiento y no trabajamos signos
- **Overflow:** El signo que queda es incoherente
- **Ejemplo:** $7(0111) + 9(1001) = 0000$. Sería $16(10000)$ pero hay desbordamiento

Acarreo: 1	1	1	1	
	0	1	1	1
+	1	0	0	1
1	0	0	0	0

Resta: Igual que suma, pero el pasamos el segundo número a negativo

Multiplicación Sin Signo:

- Multiplicar el primero por cada bit del segundo
- Añadir un 0 por cada bit
- Sumar Cada multiplicación
- **Reglas:**
 - $0 \cdot 0, 0 \cdot 1, 1 \cdot 0 = 0$
 - $1 \cdot 1 = 1$
- **Ejemplo:** $7(0111) \cdot 3(0011) = 21(10101)$

		0	1	1	1
.		0	0	1	1
		0	1	1	1
+	0	1	1	1	0
	1	0	1	0	1

Multiplicación Con Signo:

- Multiplicaciones $M \cdot Q$
- Registro S es el complemento a 2 de M
- Registro A con todo 0s y registro Q_{-1} de 1 bit a 0
- Pasos: Algoritmo de Booth
 1. Si último bit de Q (Q_0) distinto de Q_{-1}
 - Si Q_0 es 0 y Q_{-1} es 1, $A+M$ y guardo en A. Se pierde acarreo
 - Si Q_0 es 1 y Q_{-1} es 0, $A+S$ y guardo en A. Se pierde acarreo
 2. Si son iguales, no hago nada
 3. Desplazar los bits a la derecha de la siguiente manera
 - A_0 al primer bit de Q
 - Q_0 a Q_{-1}
 - Se pierde Q_{-1}
 4. Se hace hasta que se acaben los bits de Q
 5. Resultado: Concatenar A y Q

4. Ejemplo: $-7(1001) \cdot 3(0011)$

A	Q	Q_{-1}	M	S	Paso
0000	0011	0	1001	0111	
0111	0011	0	1001	0111	$Q_0 \neq Q_{-1}$, Tipo 1-0, $A+S$
0011	1001	1	1001	0111	Desplazo
0001	1100	1	1001	0111	$Q_0 = Q_{-1}$, Desplazo
1010	1100	1	1001	0111	$Q_0 \neq Q_{-1}$, Tipo 0-1, $A+M$
1101	0110	0	1001	0111	Desplazo
1110	1011	0	1001	0111	$Q_0 = Q_{-1}$, Desplazo

- Ejemplo 2: $-9(10111) * -11(10101)$

A	Q	Q_{-1}	M	S	Paso
00000	10101	0	10111	01001	$Q_0 \neq Q_{-1}$, Tipo 1-0, $A+S$
01001					Desplazo
00100	11010	1			
11011					$Q_0 \neq Q_{-1}$, Tipo 0-1, $A+M$
00110	11101	0			Desplazo
00110					$Q_0 \neq Q_{-1}$, Tipo 1-0, $A+S$
00011	01110	1			
11010					$Q_0 \neq Q_{-1}$, Tipo 0-1, $A+M$
11101	00111	0			Desplazo
00110					$Q_0 \neq Q_{-1}$, Tipo 1-0, $A+S$
00011	00011	1			Desplazo

División:

- Para divisiones Q/M
 - Registro S es el complemento a 2 de M
 - Registro A con todo 0s si Q positivo y con 1s si Q negativo
 - Pasos:
 1. Desplazó A y Q un bit a la izquierda
 - Si M y A tiene el mismo signo (primer bit es el mismo), A+S. El acarreo se pierde. Guardar en A el resultado
 - Si no, A+M. El acarreo se pierde. Guardar en A el resultado
 2. Compruebo si la operación ha tenido éxito (signo de A no cambia)
 - Si tiene éxito (no cambia), el último bit de Q pasa a ser 1
 - El último bit de Q es 0 y A vuelve a su valor anterior
 3. Se repite el proceso hasta que se acaben los bits del dividendo original
 4. Cociente en Q
 - Si los signos del divisor y el dividendo son iguales, cociente en Q
 - Si no, el cociente es el complemento a 2 de q
 5. A es el resto
 - Si M es negativo, Resto es el complemento a 2 de este número
5. Ejemplo: $-7(1001) / -3(1101)$

A	Q	M	S	Paso
1111	1001	1101	“11	
	0010			Bit a la izquierda
0010				Signo A = Signo M, A+S
1111				No hay Exito
1110	0100			Bit a la izquierda
0001				Signo A = Signo M, A+S
1110				No hay Exito
1100	1000			Bit a la izquierda
1111				Signo A = Signo M, A+S
	1001			Hay Exito, cambio Q ₀ a 1
	0010			Bit a la izquierda
0010				Signo A = Signo M, A+S
1111				No hay Exito

4.3 – Conversión

4.3.1 – Binario y Decimal

Decimal a Binario:

- **Método 1:** División
 - Dividir el número entre 2 hasta llegar a 0
 - Los restos en sentido inverso es el número en binario
- **Método 2:** Potencias de 2
 - Para 4n bits
 - Resto el valor del bit a el número:
 - Si resultado mayor o igual que 0 - Bit es 1 y mi número ahora es el resultado
 - Si resultado menor que 0 - Bit es 0
 - Aplicar hasta llegar a restar 1
 - Sirve para los decimales
- **Método 3:** Método para decimales
 - Multiplicar número por 2. Bit es la parte entera del resultado
 - Repetir hasta tener suficientes bits o hasta que desaparezca parte decimal
- **Ejemplo 1:** 107 a binario

Método 1:	Método 2
$107/2 = 53r1$ $53/2 = 26r1$ $26/2 = 13r0$ $13/2 = 6r1$ $6/2 = 3r0$ $3/2 = 1r1$ $1/2 = 0r1$ $107_{10} = 01101011_2$	$107 - 128 < 0 \rightarrow \text{Bit} = 0$ $107 - 64 = 43 > 0 \rightarrow \text{Bit} = 1$ $43 - 32 = 11 > 0 \rightarrow \text{Bit} = 1$ $11 - 16 < 0 \rightarrow \text{Bit} = 0$ $11 - 8 = 3 > 0 \rightarrow \text{Bit} = 1$ $3 - 4 < 0 \rightarrow \text{Bit} = 0$ $3 - 2 = 1 > 0 \rightarrow \text{Bit} = 1$ $1 - 1 = 0 \rightarrow \text{Bit} = 1$ $107_{10} = 01101011$

- **Ejemplo 2:** 0.6875 a binario

Número:	Decimal:	Bit:	
0.6875	0,375	1	$0,6875 - 0,5 = 0,1875 > 0 \rightarrow \text{Bit} = 1$
0,375	0,75	0	$0,1875 - 0,25 < 0 \rightarrow \text{Bit} = 0$
0,75	0,5	1	$0,1875 - 0,1875 = 0 \rightarrow \text{Bit} = 1$
0,5	0	1	$0,0625 - 0,0625 = 0 \rightarrow \text{Bit} = 1$

$$0,6875_{10} = 0.1011_2$$

Binario a Decimal: Aplicar Teorema Fundamental de la Numeración

- **Ejemplo:** 01101011_2 a decimal
 - $01101011_2 = 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 1+2+8+32+64 = 107_{10}$

4.3.2 – Octal y Decimal

Decimal a Octal: Mismo método que para binario. **Método 1**

- **Ejemplo:** 1992_{10} en octal
 - $1992/8 = 249r0$
 - $249/8 = 31r1$
 - $31/8 = 3r7$
 - $3/8 = 0r3$
 - $1992_{10} = 3710_8$

Octal a Decimal: Aplicar Teorema Fundamental de la Numeración

- **Ejemplo:** 3710_8 a decimal
 - $3 \cdot 8^3 + 7 \cdot 8^2 + 1 \cdot 8^1 + 0 \cdot 8^0 = 1992_{10}$

4.3.3 – Hexadecimal y Decimal

Decimal a Hexadecimal: Mismo método que para binario. **Método 1**

- **Ejemplo:** 1992_{10} en decimal
 - $1992/16 = 124r8$
 - $124/16 = 7r12$
 - $7/16 = 0r7$
 - $1992_{10} = 7C8_{16}$

Hexadecimal a Decimal: Aplicar Teorema Fundamental de la Numeración

- **Ejemplo:** $7C8_{16}$ a Decimal
 - $7 \cdot 16^2 + 12 \cdot 16^1 + 8 \cdot 16^0 = 1992_{10}$

Tema 5 – Funcionamiento de Computador

5.1 – Introducción

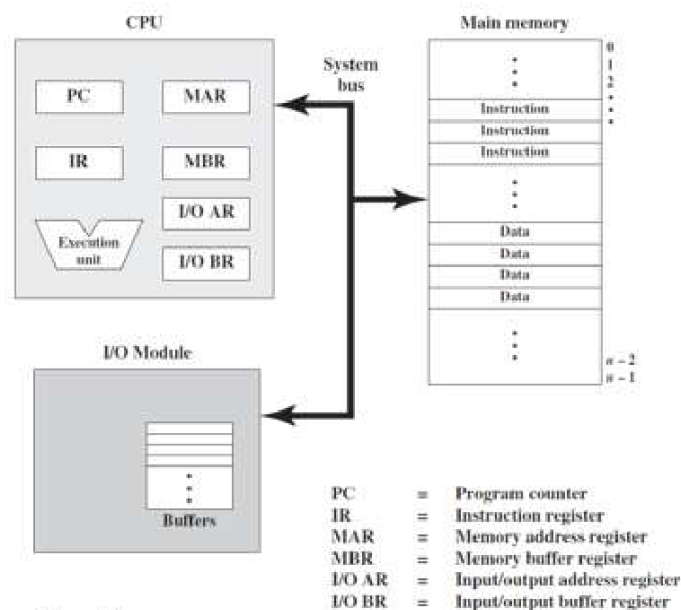
Hardwired Program:

- Programa Cableado
- Función programada mediante circuitos electrónicos
- **Características:**
 - Los Mismos datos de entrada dan la misma salida
 - Solo puede ejecutar la función para la que ha sido programado
 - Otra función implica otro circuito

Programación Software:

- Distinción entre las funciones que se pueden ejecutar y el problema que se lleva a cabo
- **Características:**
 - Conjunto Fijo de funciones lógicas y aritméticas
 - Conjunto de instrucciones que modifican el programa
 - Conjunto de señales de control que decodifica cada instrucción
 - Mismos datos de entrada no implican misma salida
- **Basada en la Arquitectura Von Neumann:**
 - Datos y instrucciones se almacenan en memoria de lectura/escritura
 - Contenidos de la memoria se identifican mediante una dirección
 - Ejecución secuencial

5.2 – Estructura General del Ordenador Programable



El CPU intercambia información mediante registros

Memoria: Conjunto de posiciones identificadas mediante una dirección

Contenido escrito en binario y puede ser interpretado como dato o instrucción

Módulo E/S: Transfiere datos desde un dispositivo externo a la CPU

Contiene registros propios

Transferencia a través del bus del sistema

Registros de la CPU:

- **MAR:** Especifica la próxima dirección de memoria
- **MBR:** Almacena el dato leído
- **IOAR:** Dirección del dispositivo E/S
- **IOBR:** Almacena el dato a transferir al dispositivo E/S
- **PC:** Dirección de la siguiente instrucción a ejecutar
- **IR:** Contiene la siguiente instrucción a ejecutar

5.3 – Ejecución

Fetch-Execute Cycle (Ciclo de Captación y ejecución)

Ciclo de Captación:

- Se extrae la instrucción indicada en el PC y se almacena en el IR.
 - Se copia la dirección en el PC en el MAR.
 - Mediante el bus de direcciones, llevamos esa dirección a la memoria y accedemos a la dirección de memoria correspondiente
 - Mediante el bus de datos, llevamos el contenido de esa dirección de memoria (la instrucción) de vuelta al CPU y la guardamos en el MBR.
 - Copiamos la instrucción en el IR
- Incrementamos en PC para poder acceder a la siguiente instrucción

Ciclo de Ejecución:

- Se interpreta la instrucción
- **Tipos de Instrucciones:**
 - Transferir Datos entre CPU y memoria
 - Transferir Datos en E/S y CPU
 - Aplicar operación aritmético lógica
 - Alterar el valor de PC

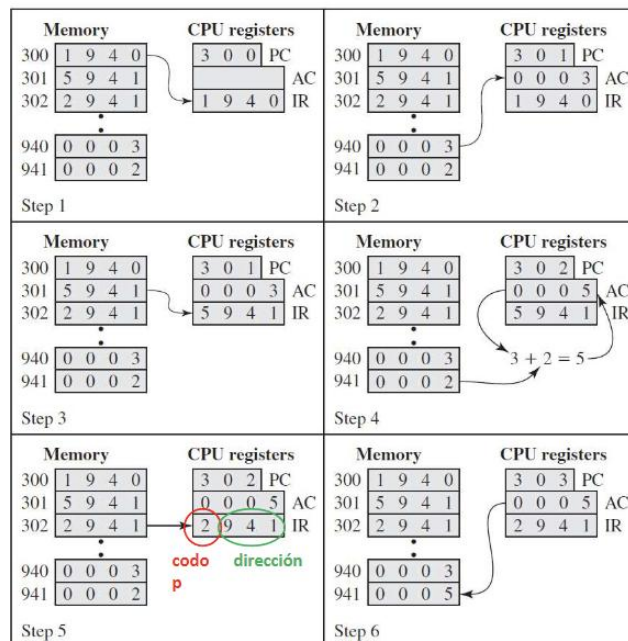
5.4 – Ejemplo de Una Ejecución

Codop: Instrucciones al ejecutar

Ejemplos:

- 1: Cargar AC desde memoria
- 2: Guardar AC en memoria
- 5: Suma a AC un dato de la memoria

Formato: Codop Datos/Dirección

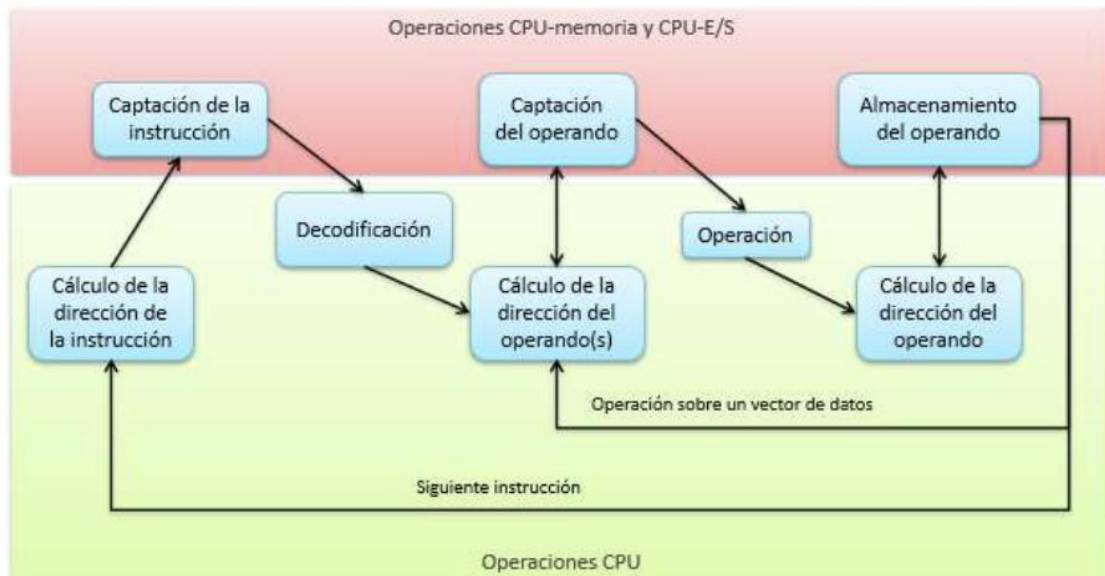


- 1) Como el PC está en el 300, cargar el contenido de la dirección 300 en el IR.
- 2) Codop: 1, Dirección 940.
 - a) Cargar en AC el contenido de la dirección 940
 - b) Aumento uno el PC
- 3) Cargar el contenido de la dirección 301 en el IR
- 4) Codop: 5, Dirección: 941
 - a) Sumo el contenido del 941 en AC
 - b) Aumento el PC
- 5) Cargar el contenido de la dirección 302 en el IR
- 6) Codop: 2, Dirección: 941
 - a) Guardar el contenido del AC en el 941
 - b) Aumento el PC

5.5 – Diagrama de Estados de Un Ciclo

Diagrama de Estado del Ejemplo:

Diagrama de estados de un ciclo de instrucción:



5.6 – Ciclo de Interrupción

Dentro del ciclo de instrucción

Interrupción: Mecanismo que interrumpe el funcionamiento normal del CPU para que esta gestione una incidencia

Incidencia:

- Operación de E/S
- Temporización
- Fallo de Software o Hardware

Más comunes de E/S ya que los dispositivos suelen operar más lentamente que el CPU. Evita que el CPU se quede esperando a estos dispositivos.

Cada vez que acaba el ciclo de ejecución se ve si hay interrupciones pendientes

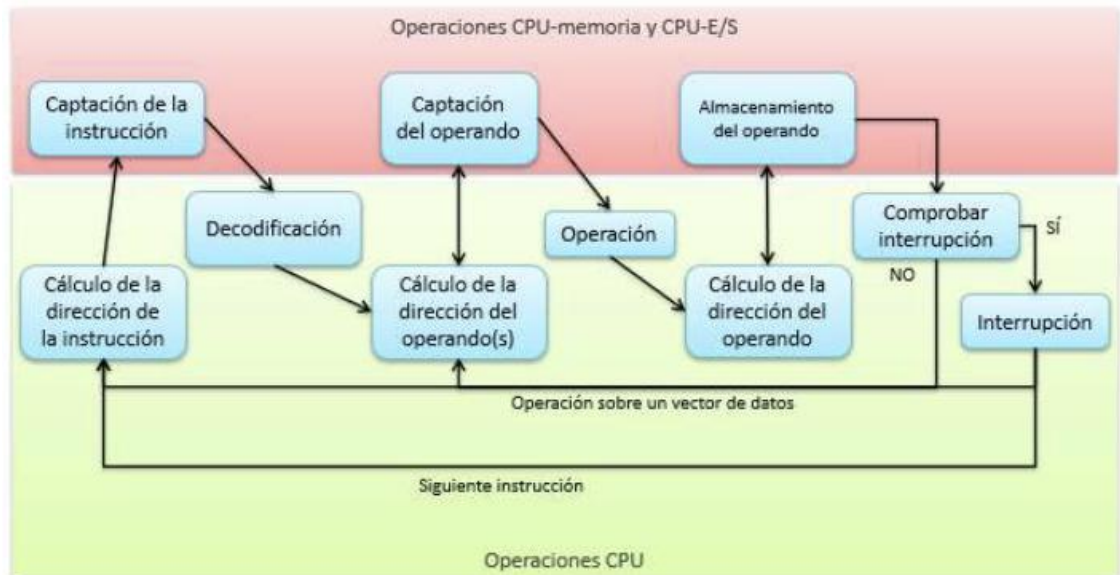
Ciclo de Interrupción:

- 1) Ciclo de Captación
- 2) Ciclo De Ejecución
- 3) Si hay Interrupción pendiente:
 - a) Comienza el ciclo de interrupción
 - b) Suspende la ejecución del programa y almacena su contexto
 - c) Mueve el PC al inicio del programa de gestión de interrupciones
- 4) Si no hay. Paso al siguiente ciclo

Ventajas de las Interrupciones:

- CPU puede seguir ejecutando mientras los dispositivos E/S hacen su trabajo
- Mejoría de la eficiencia de la CPU

Diagrama de estados de un ciclo de instrucción con interrupciones:



6 – El Procesador

6.1 – Lenguaje Maquina

Definiciones:

- **Programa Fuente:** Archivo de texto que contiene el código escrito. No puede ser ejecutado directamente, sino compilado o interpretado antes.
- **Lenguaje Ensamblador:** Lenguaje de programación de bajo nivel que permite escribir código fácilmente traducido a lenguaje máquina. Permite un control más fino del CPU, pero requiere conocimiento profundo de la arquitectura y memoria.
- **Ensamblador:** Traduce lenguaje ensamblador a lenguaje máquina
- **Programa Objeto:** Archivo generado a partir del programa fuente que contiene el código traducido en formato ejecutable
- **Lenguaje Máquina:** Instrucciones que ejecuta el procesador. Programas directamente en código binario
- **Lenguaje de alto nivel:** Lenguaje de programación con sintaxis cercana al lenguaje humano. Basada en conceptos abstractos como las variables y las estructuras de control y flujo

6.3.1 – Instrucciones Máquina

Órdenes que el CPU puede ejecutar de manera directa

Escritas en binario

Permite:

- Cargar y almacenar valores
- Realizar operaciones aritméticas lógicas
- Saltar a diferentes zonas de código

Estructura:

- 16 bits
- Codop (0-3): Código binario que define la operación a realizar
- Operando 1(3-9)
- Operando 2(9-15)
- **Operandos pueden ser destino o origen. Pueden ser:**
 - Memoria
 - Registros
 - Dispositivos E/S
- Repertorio dependiente del CPU

Tipos de Instrucciones:

- **Procesamiento de Datos:** Operaciones Aritméticas y Lógicas
- **Almacenamiento de Datos:** Instrucciones de memoria
- **Transferencia de Datos:** Instrucciones de E/S
- **Control de Flujo:** Bifurcaciones, condiciones, iteraciones
- Cada instrucción tiene su codop y su nemónico.
- **Ejemplo:** 0001 ADD → Sumar

Operandos:

- Direcciones de Memoria
- **Números:** Representados en binario, decimal y hexadecimal
- **Caracteres:** Representado mediante ASCII (7 bits)

6.2 – Micro Z80

Zilog Z80(1976)

Arquitectura de 8 bits

6.3.1 – Registros

Área de memoria interna donde se almacena valores temporales

Más rápidos que la RAM externa

Cada registro tiene funciones específicas y pueden ser leídos y escritos en ciclos de reloj por el procesador

El Z80 tiene 16 registros de 1 byte y 4 de 2 bytes.

De estos 20, 12 son de propósito general y 8 dedicados

Registros de Propósito General:

- Sirven para almacenar datos sin tener que referencia a la memoria
- **2 juegos:** Normales y '. Solo se puede acceder a un juego a la vez
- Se pueden emparejar para trabajar con datos de 16 bits
- **EXX:** Cambia de juego
- B, C, D, E, H, L y sus versiones primas

Registros Dedicados:

- A, A', F, F', PC, IX, IY, SP
- **A:** Acumulador
 - Usando en la mayoría de las instrucciones aritméticas lógicas
- **F:** Registro de Indicadores (Flags)
 - Cambia dependiendo de la última operación sobre A
 - Indica datos como el overflow, carry, si es 0.
 - **Bit 7(S):** Signo
 - **Bit 6(Z):** Si es uno es que el resultado es 0
 - **Bit 5 y 3:** No se usan
 - **Bit 4(H):** Indica si hay acarreo en los 4 primeros bits
 - **Bit 2(P/V):** V = 1 → Overflow, P = 1 → Paridad de unos
 - **Bit 1(N):** Indica si la operación es una resta
 - **Bit 0 (C):** Acarreo
- **PC:** Contador de programa
 - 16 bits ya que se relaciona con la memoria
 - Contiene la dirección de memoria de la siguiente instrucción a ejecutar
 - Se modifica automáticamente
- **IX e IY:**
 - 16 bits
 - Se usan con operaciones de memoria
 - Sirven como punteros a direcciones de memoria
- **SP:**
 - Dirección del último dato almacenado en la pila
 - Cuando se almacena un valor en la pila. El valor del SP decrementa por 1
 - Cuando se lee un valor de la pila. Se incrementa 1

6.3.2 – Pila

LIFO → Last In First Out

Parte de la memoria principal

Cada nuevo elemento se introduce en la cima y se saca el primero

No se puede acceder a cualquier dirección de memoria dentro de ella, solo a la que apunta el SP

6.3 – Ensamblador Z80

Estructura:

- **Instrucciones:**

Etiqueta	delim	Operación	delim	Operandos	delim	Comentarios
----------	-------	-----------	-------	-----------	-------	-------------

- **Etiqueta:** Nombre único que identifica a una línea unívoca del programa
- **Operación:** Nemónico del codop con la instrucción a ejecutar
- **Operando(s):** Operando la operación. Si es doble, el primer es destino y el segundo el origen
- **Comentarios:** Comienza por ;. Opcional. Explica la línea
- **Programa:**
 - **Directiva de Comienzo:** Indica en que dirección de memoria esta la primera instrucción
 - **Etiqueta de Inicio:** Marca la dirección de memoria de inicio
 - **Directiva de Fin:** Indica cuando acaba el programa

Representación:

- **Para decimal:** El numero solo. Ejemplo: 34
- **El Hexadecimal:** Añadir H al final. Si empieza por una letra añado un 0 delante. Ej. 0A4H
- **Binario:** Añado B al final. Ejemplo: 100010B

Direccionamiento:

- **Extendido:** Operando en la dirección de memoria donde está el dato. Ej: (0C745H)
- **Indirecto:** El operando es un par de registro que contiene la dirección donde está el dato. Ejemplo: (HL)
- **Indexado:** Operando es un registro de 2 bytes más un desplazamiento
- **A través de registro:** Operando es un registro
- **Implícito:** No hay operandos. Actúa sobre registro específico. Ejemplo: CPL
- **Inmediato:** El operando es un dato
- **Inmediato Extendido:** Se almacena en 2 bytes